Take-Home Project

React Native + TypeScript App

Objective

Build a simple React Native mobile app using TypeScript that accesses a REST API and renders the result in a list. The app should allow the user to select an item from the list and view more details on a second screen.

Project Requirements

1. GitHub Repository

Create a public GitHub repository under your GitHub account for this project. Email us the GitHub repository link as soon as possible.

Your repository should include:

- The full React Native source code
- A clear README.md
- Instructions for running the app locally
- Any assumptions or tradeoffs you made

2. Technology Requirements

Use the following technologies:

- React Native
- TypeScript

You may use either Expo or React Native CLI. Expo is recommended unless you have a specific reason to use the React Native CLI.

You may use common open-source libraries such as:

- React Navigation
- Axios or fetch
- React Query, SWR, or a similar data-fetching helper
- UI libraries, if desired

Please avoid using a full starter template that already implements most of the requested functionality.

3. API Requirement

The app should fetch public AggieFeed activity data from the following REST API:

```
https://aggiefeed.ucdavis.edu/api/v1/activity/public?s=0&l=25
```

The API should return up to 25 public AggieFeed activities. Your app should fetch this data when the main screen loads.

App Requirements

Main Screen

The main screen should display a list of activities returned by the API. Each list item should show:

```
title
actor.displayName
```

The list should be implemented using an appropriate React Native list component, such as FlatList.

The main screen should also handle:

- Loading state while the API request is in progress
- Error state if the API request fails
- Empty state if no activity data is returned

Detail Screen

When the user taps an activity in the list, the app should navigate to a detail screen. The detail screen should show the selected activity's:

```
title
actor.displayName
object.objectType
published
```

The detail screen should allow the user to return to the main list using normal mobile back navigation.

TypeScript Expectations

Define appropriate TypeScript types or interfaces for the activity data returned by the API. At minimum, your types should account for:

```
title
actor.displayName
object.objectType
published
```

The app should not rely heavily on any. If some API fields may be missing or undefined, handle that gracefully in the UI.

User Experience Expectations

The app does not need to be visually complex, but it should be usable and readable. Please pay attention to:

- Clear list item layout
- Readable spacing and typography
- Obvious tap targets
- Graceful handling of missing data
- Basic mobile usability on iOS and/or Android

README Requirements

Your README.md should include:

- Project overview
- Setup instructions
- How to run the app
- Any required environment setup
- Libraries used
- Known issues or limitations

Example commands may include:

```
npm install
npm start
```

or

```
yarn install
yarn expo start
```

Evaluation Criteria

Category	What we will evaluate
Functionality	App fetches data from the AggieFeed API; main screen displays a list of activities; detail screen displays the selected activity details; navigation works correctly.
Code Quality	Clear project structure; good React Native practices; appropriate use of TypeScript; readable, maintainable code; minimal unnecessary complexity.
Error Handling	Loading state is handled; API errors are handled; missing or unexpected data does not crash the app.
User Experience	The app is easy to use, readable, and intuitive.
Documentation	README is clear and complete; setup steps are accurate; assumptions and limitations are documented.

Optional Enhancements

These are not required, but may help demonstrate additional skill:

- Pull-to-refresh on the activity list
- Basic unit tests
- Basic component tests
- Date formatting for the published field
- Improved visual styling
- Accessibility labels
- Support for both light and dark mode
- Retry button on API error
- Use of a data-fetching library such as React Query

Submission Instructions

When complete, email us the link to your public GitHub repository. Your submission should include enough documentation for us to clone the repository and run the app locally.